DAWAN Paris 11,rue Antoine Bourdelle, 75015 PARIS
DAWAN Nantes 32, Bd Vincent Gâche, 5e étage - 44200 NANTES
DAWAN Lyon Bt Banque Rhône Alpes, 2ème étage – 235 cours Lafayette 69006 LYON



Formation Python: Design Pattern, Temps réel et Tests

Plus d'info sur http://www.dawan.fr ou 0810.001.917

Formateur: Matthieu LAMAMRA

Présentation de la formation

Qui suis-je? « mon parcours » « mes compétences »

Exprimer votre besoin:

- Le contexte **IT**
- Votre environnement de travail
- Vos **objectifs** pour cette formation.

DAWAN - Reproduction interdite

Objectifs: Design Patterns

Découvrir les **fonctionnalités avancées** du langage

Déployer des modules pythons tiers

Implémenter en python les principaux **patrons de conceptions**



Avec ces acquis vous serez en mesure de :

- Maîtriser les subtilités du code
- Gérer les dépendances de vos projets
- Standardiser les cas d'utilisations python

DAWAN - Reproduction interdite

Objectifs: Temps réel et Tests

Comprendre le **mode d'exécution** de Python

Lancer et faire communiquer des **threads concurrents ou des processus en parallèle**

Tester une application Python dans un **environnement Agile**



Avec ces acquis vous serez en mesure de :

- Construire une **application multitâches**
- Lancer des **tests unitaires et « E2E » automatisés**

DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Fonctions Lambdas

- Fonctions éphémères, sans nom, constituées d'une seule instruction, à usage unique
- Issues du paradigme de programmation fonctionnelle et du lambda calcul
- Composition :
  - le mot clé lambda
  - une ou plusieurs variables liées, définis comme des paramètres de fonctions
  - un corps composé d'une expression contenant des variables liées ou libres
- Les lambda sont affectables (!= Pythonique)

```
y = 4 # variable libre
l_func = lambda x, z, *args:  x+ y + z # x, z variables liées
l_func(1, z=3)
```

Fonctionnalités Avancées

- Fonctions Lambdas

- Le corps des lambdas n'acceptent pas les instructions :

  - return
  - pass
  - assert
  - raise
  - …

- On peut appeler directement une lambda après sa déclaration

```
(lambda x: x * x)(3)

9
```

Fonctionnalités Avancées

- Fonctions Lambdas : utilisations « pythoniques »

- Dans des fonctions standards de python :
  - map(lambda x: x +1, […]) : applique une lambda à tous les éléments d'une séquence et retourne la séquence des résultats
  - filter(lambda x : 'thing' in x, [...]) : filtre une liste selon le résultat de la lambda
  - functools.reduce(lambda acc, x : acc + x, […]) : transforme une séquence en scalaire en opérant une lambda sur des éléments de proche en proche

- Dans des fonctions de tri

```
ids = ['id1', 'id2', 'id30', 'id3', 'id22', 'id100']
sorted_ids = sorted(ids, key=lambda x: int(x[2:]))
['id1', 'id2', 'id3', 'id22', 'id30', 'id100']
```

Fonctionnalités Avancées

- Fonctions Lambdas : utilisations non « pythonique »

➢ Dans les classes :

```
class Person:
    _name = ‘’
    def __init__(self, name):
        self._name= name

    name=property(lambda self: getattr(self, '_name'),
                  lambda self, value: setattr(self, '_name', value))
```

E731 do not assign a lambda expression, use a def

Fonctionnalités Avancées

- Les itérables
  - Sont des classes qui implémentent les méthodes implicites **__init__, __iter__ et __next__**
  - Peuvent être utilisés avec iter() et next(), et dans les boucles for, qui appellent ces fonctions
  - L'exception StopIteration stoppe l'iterable, elle est capturée par la boucle for

```
class PowTwo:
  def __init__(self, max = 0):
    self.max = max
  def __iter__(self):
    self.n = 0
    return self
  def __next__(self):
    if self.n <= self.max:
      result = 2 ** self.n
      self.n += 1
      return result
    else:
      raise StopIteration
```

```
In [2]: a = PowTwo(4)

In [3]: i = iter(a)

In [4]: next(i)
Out[4]: 1

In [5]: next(i)
Out[5]: 2

In [6]: next(i)
Out[6]: 4
```

```
In [10]: for pt in PowTwo(5):
    ...:     print(pt)
    ...:
1
2
4
8
16
32
```

Fonctionnalités Avancées

- Les itérables : relation avec les séquences

  - Les objets iterables prennent en charge des fonctionnalités accessibles aux séquences (indexables)
  - Utilisables avec des builtins python comme sum, max, map, zip...
  - Les itérables gèrent l'unpacking (affectable dans un tuple implicite)

```
sum(PowTwo(4))
max(PowTwo(4))
map(str, PowTwo(4))
…
a, b, c, d, e = PowTwo(4)
first, *others = PowTwo(4)
```

Fonctionnalités Avancées

- Itérables, itérateurs

  - La fonction iter retourne un itérateur à partir d'un itérable. c'est cet itérateur qui est itéré dans une boucle et par la fonction next

  - Un itérateur se consomme quand il est parcouru

  - L'itérateur peut être l'objet instance de la classe itérable, ou un objet d'une autre classe itérable

```
class MyRange:
    def __init__(self, size):
        self.size = size

    def __iter__(self):
        return MyRangeIterator(self)
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Le module itertools

  - Le module itertools de la librairie standard met à disposition des développeurs des fonctions retournant des iterateurs ou manipulant des iterateurs
  - Des itérateur infinis : count(), cycle(), repeat()
  - Des modifications d'itérateurs : accumulate(), chain(), islice()
  - Des opérateurs combinatoires sur des itérateurs : product(), permutations(), combinations()

https://docs.python.org/3/library/itertools.html

Fonctionnalités Avancées

- Générateurs : définitions

- Les générateurs sont des itérables et des itérateurs construits à partir de fonctions, avec utilisation du mot clé yield

- Le générateur stricto sensu est la valeur retournée par l'appel de la fonction

- Cette valeur en paramètre de next, ou dans une boucle, exécute le code de la fonction jusqu'à la plus proche instruction yield et retourne la valeur en paramètre ou None

```
def function():
    yield 'first value'
    yield
    yield 3

gen = function()
next(gen) # 'first value'
next(gen) # None
next(gen) # 3
next(gen) # Error : StopIteration
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Générateurs : utilisations usuelles

- l'intérêt principal des générateurs réside dans un gain en mémoire dans l'itération par rapport à d'autres itérables comme les listes, puisque les valeurs itérées y sont calculées de proche en proche. Les générateurs peuvent même être infinis

```
# fonction génératrice
def fibonacci(n, a=0, b=1):
    for _ in range(n):
    yield a
    a, b = b, a + b

# boucle
for i in fibonacci(10) :
    print(i)

# conversion du générateur en liste
list(fibonacci(10))
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Générateurs : la méthode send

- Les générateurs disposent d'une méthode send qui permet de communiquer avec eux après leur création, en modifiant la succession des valeurs retournées.

- L'appel à send() exécute le code de la fonction génératrice depuis une instrution yield jusqu'à la prochaine, comme le ferait next(). Il est ainsi nécessaire d'avoir exécuté au moins une fois le générateur.

- Le paramètre de send est disponible comme valeur de retour de l'instruction yield, qui devient affectable

```
def queue(*args):
    elems = list(args)
    while elems:
        new = yield elems.pop(0) # l'exécution s'interrompt avant l'affection
        if new is not None:
            elems.append(new)

q = queue('a', 'b', 'c')
next(q)  # a
q.send('d') # b et rajoute d à la fin
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Générateurs : la méthode throw

- Les générateurs disposent d'une méthode throw qui permet d'envoyer une exception dans la fonction génératrice, capturable par celle ci.

- Si l'exception et capturée, le générateur exécute un code spécifique (except) et renvoie la prochaine valeur comme avec next()

- Sinon l'exception s'affiche et le générateur est terminé

```
def generator_function():
    for i in range(5):
        try:
            yield i
        except ValueError:
            print('Something goes wrong')

gf = generator_function()
next(gf)  # 0
gf.throw(ValueError) # something goes wrong, puis 1
netxt(gf) # 2
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Générateurs : la méthode close

- Les générateurs disposent d'une méthode close qui permet d'envoyer une exception GeneratorExit dans la fonction génératrice, capturable par celle ci.

- Si l'exception et capturée, le générateur exécute un code spécifique (except) qui permet de terminer le générateur (break, return), ou un autre code qui générera une RuntimeError et le générateur continuera

- Sinon l'exception s'affiche et le générateur est terminé

```
def test_close():
    for i in range(5):
        try:
            yield i
        except GeneratorExit:
            break

tc = test_close()
next(tc)  # 0
tc.close
next(tc) # StopIteration
```

Fonctionnalités Avancées

- Générateurs : l'instruction yield from

- l'intruction yield from permet de déléguer l'exécution d'un générateur à un autre générateur interne au premier

- En fonction de l'étape d'exécution, un appel à send pourra concerner uniquement le générateur interne

```
def add_queue():
    yield 0
    yield from queue(1 , 2, 3)
    yield 4

aq = add_queue()
next(aq)  # 0
next(aq) # 1
aq.send(5) # ajoutera 5 la suite de 3 et non 4
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Générateurs en intension

➢ On peut créer des générateurs relativement simples de manière explicite comme pour les listes en intension, mais utilisant des parenthèses () au lieu des cochets []

```
squares = (x**2 for x in range(10))
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Décorateurs : Définition

- Les décorateurs désignent un mécanisme (design pattern) de python permettant de modifier extérieurement le comportement d'un callable python, i.e fonctions et classes

- En général, un décorateur en python est une fonction qui prend en paramètre une fonction et qui retourne une fonction

- On peut utiliser le caractère @ suivi de la fonction décoratrice en entête de la définition de la fonction décorée

```
def decorator(f): # fonction décoratrice
    print(f.__name__)
    return f # cas simple : la fonction retournée est celle en paramètre

@decorator
def add(a, b) : # fonction décorée
    return a + b

# Le décorateur  est appliqué à la définition de la fonction décorée
# Equivalent à : add = decorator(add)
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Décorateurs : Cas général

- La fonction retournée par le décorateur est le plus souvent une fonction implicite au décorateur, qui contient l'appel à la fonction décorée en paramètre du décorateur, et des instructions modifiant ou augmentant le comportement de la fonction décorée

```
def print_decorator(function):
    def new_function(*args, **kwargs): # *args et **kwargs pour la généricité
        print('Appel de la fonction {} avec args={} et kwargs={}'.format(
            function.__name__, args, kwargs))
        ret = function(*args, **kwargs)
        print('Retour: {}'.format(ret))
        return ret
    return new_function
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Décorateurs paramétrés

  - Un décorateur peut posséder des paramètres.

  - Un Décorateur paramétré se comporte comme un décorateur sur un décorateur. Il retourne une fonction implicite qui est elle même le décorateur de la fonction décorée

```
def param_print_decorator(before, after): # Décorateur paramétré
    def decorator(function): # Décorateur
        def new_function(*args, **kwargs): # Fonction qui remplacera la fonction décorée
            print(before.format(function.__name__, args, kwargs))
            ret = function(*args, **kwargs)
            print(after.format(ret))
            return ret
        return new_function
    return decorator

@param_print_decorator('call {} with args({}) and kwargs({})', 'ret={}')
def test_func(x):
    return x

# Equivalent à :                test_func = param_print_decorator(before, after)(test_func)
```


DAWAN - Reproduction interdi

Fonctionnalités Avancées

- Décorateurs paramétrés

- Un décorateur peut posséder des paramètres.

- Un Décorateur paramétré se comporte comme un décorateur sur un décorateur. Il retourne une fonction implicite qui est elle même me décorateur de la fonction décorée

```
@param_print_decorator('call {} with args({}) and kwargs({})', 'ret={}')
def test_func(x):
    return x

def param_print_decorator(before, after): # Décorateur paramétré
    def decorator(function): # Décorateur
        def new_function(*args, **kwargs): # Fonction qui remplacera la fonction décorée
            print(before.format(function.__name__, args, kwargs))
            ret = function(*args, **kwargs)
            print(after.format(ret))
            return ret
        return new_function
    return decorator
# Equivalent à :                test_func = param_print_decorator(before, after)(test_func)
```

Fonctionnalités Avancées

- Décorateurs : introspection

- La fonction retournée par le décorateur redéfinit la fonction décorée, notamment les métadonnées d'introspection, notamment pour la fonction help()

- Le décorateur wraps du module functools, avec la fonction décorée en paramètre, permet d'associer ces métadonnées avec la fonction retournée du décorateur

```
import functools

def decorator(f):
    @functools.wraps(f)
    def decorated(*args, **kwargs):
        return f(*args, **kwargs)
    return decorated
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Exemple de décorateur : méthodes de classes

- En python, les attributs crées à la définition de la classe sont des attributs de classes
- Les attributs crées dans __init__ ou autre méthodes sont des attributs d'instance
- En POO, une méthode statique est une méthode exécutée directement par la classe, sans instanciation
- En python, on parlera de méthode de classe
- Par défaut, toutes les méthodes définies dans une classe sont des méthodes d'instance
- Le décorateur @classmethod autorise la méthode à être appelable par la classe. Le premier argument est alors par convention cls, et fait référence à la classe
- En python, une méthode statique est une méthode classe sans aucun accès aux attributs de la classe, générée par le décorateur @staticmethod

```
class CountInstances():
    nb_instances = 0

    def __init__(self) :
        CountInstances.nb_instances += 1 # opération sur l'attribut de classe

    @classmethod
    def count_instances(cls):
        return cls.nb_instances
```

Fonctionnalités Avancées

- Contextes : Le mot clé with:
  - Permet d'écrire un bloc qui **gère des opérations avant la définition de l'entête et après l'exécution du bloc**

```
In [1]: with open("fichier_test.txt", "r") as f:
   ...:         text = f.read()
   ...:
```

```
In [9]: with conn:
   ...:     cur = conn.cursor()
   ...:     cur.execute("SELECT * FROM proprietaires")
   ...:     rows = cur.fetchall()
   ...:     print(rows)
   ...:
[('1233 DC 81', 'Martin', 'Rémy'), ('1213 FC 98', 'Benhamicha', 'Hakim')]
```

Fonctionnalités Avancées

- Contextes : fonctionnement

- Une classe peut être utilisée avec with si elle implémente les méthodes spéciales __enter__() et __exit__()

```
class MyContextManager():
    def __enter__(self):
        print "Avant"
    def __exit__(self, type, value, traceback):
        # paramètres automatiquement passées à __exit__()
        # en cas de le vée d'exception dans le bloc
        print "Après"

with MyContextManager():
    truc()
```

Fonctionnalités Avancées

- Contextes : mot clé as
  - La valeur de retour de la méthode __enter__() est disponible dans le bloc de contexte par association au mot clé as

```
from pymongo import MongoClient

class MongoDBManager():
    def __init__(self, hostname, port):
        self.hostname = hostname
        self.port = port

    def __enter__(self):
        self.connection = MongoClient(self.hostname, self.port)
        return self # l'instance de connection sera utilisable dans le bloc du contexte

    def __exit__(self, exc_type, exc_value, exc_traceback):
        self.connection.close()

# utiliser le contexte de connection
with MongoDBManager('localhost', '27017') as mongo:
    collection = mongo.connection.SampleDb.test
```

Fonctionnalités Avancées

- Contextes : gestion des exceptions

```
class Ctx:
  def __enter__(self):
    pass

  def __exit__(self, x_type, x_value, x_trace):
    if x_type:
      print(x_type, x_value)
      return True

with Ctx():
  3/0
```

- __enter__ fonctionne par défaut comme un bloc try englobant le bloc du contexte
- Une exception dans le bloc est récupérée par __exit__ grâce à trois paramètres
- l'exception sera capturée si __exit__ retourne une valeur non fausse

DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Contextes réentrants :

```
class Cd :
  def __init__(self, path):
    self.path = path
    self.old_paths = []

  def __enter__(self):
    self.old_paths.append(os.getcwd())
    os.changedir(self.path)
    print(os.getcwd())

  def __exit__(self):
    os.changedir(self.old_paths.pop())

with Cd('~/dir1):
  with Cd('./dir2'):
    pass
print(os.getcwd())
```

- Un contexte est réutilisable si la resource qu'il exploite n'es pas libérée par la fonction __exit__
- Un contexte est réentrant s'il est réutilisable imbriqué dans lui même

DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Le décorateur contextlib.contextmanager :

```
@contextmanager
def managed_resource():
    # enter
    resource = "hello"
    try:
        # "as" value
        yield resource
        # block
        int(resource)
    except Exception as e:
        print(e)
    finally:
        # exit
        del resource

with managed_resource() as r:
    print(r.upper())
```

- Le décorateur contextmanager permet d'utiliser un générateur comme contexte
- Le générateur répartie les actions du « enter » et du « exit » de part et d'autre de l'instruction yield
- La valeur « rendue » par yield correspond au mot clé « as » du contexte

Fonctionnalités Avancées

- Les métaclasses : la « fonction » type

- type(var) renvoie le nom de la classe de l'instance affectée à var « Tout est objet »

- type(cls_name, bases, attrs_dict) a comme valeur de retour une classe, avec :
  - cls_name : chaine de caractère contenant le nom de la nouvelle classe
  - bases : tuple contenant les classes dont cls_name hérite
  - attrs_dict : dictionnaires des attributs et méthodes

- En python, type et toute classe héritant de type sont appelées métaclasses, soit des classes dont les instances sont des classes

```
# équivalence
Class B(A) :
    a = ‘’

B2 = type(‘B2’, (A,), {‘a’ : ‘’})

type(B) # type
type(B2) # type
B2.__class__ # type : équivalence type – classe pour python > 3.0
```


DAWAN - Reproduction interdite

Fonctionnalités Avancées

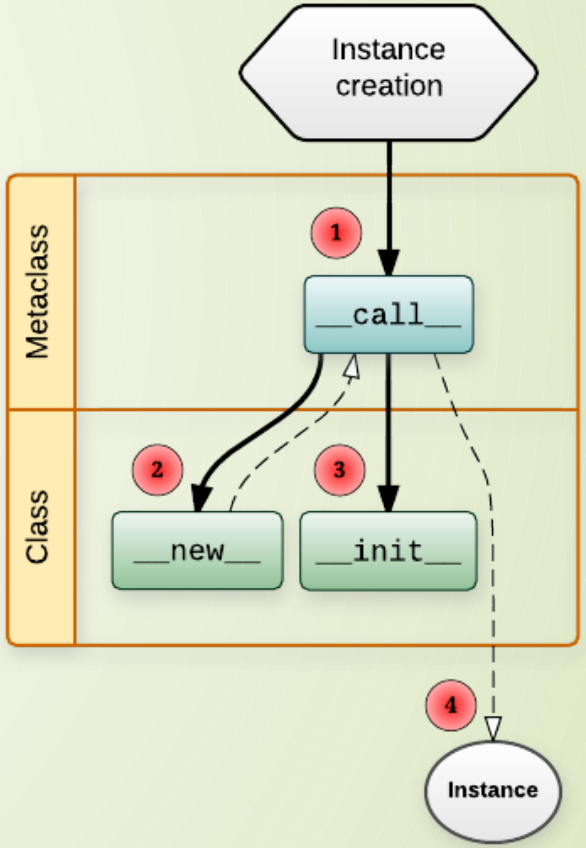
- Les métaclasses : sous le capot

- Lorsqu'un objet est instancié, la séquence suivante est exécutée :
  - Appel de la méthode __call__() de la métaclasse qui fait elle même
  - Appel au constructeur __new__() de la classe : création de l'instance
  - si __new__ retourne qqch __call__ fait encore
  - Appel à l'initialiseur __init__() de la classe : création des attributs d'instance
  - __call__ retourne l'instance

- __call__ est systématiquement appelé par l'opérateur ()

- Si __new__ n'est pas présent dans la classe, le constructeur de la métaclasse sera appelé => la classe « hérite » de sa métaclasse

- 99 % des objets python sont créés par type.__new__



DAWAN - Reproduction interdite

Fonctionnalités Avancées

- Créer une métaclasse

- type.__new__ n'est pas réaffectable en python
- Pour créer une métaclasse, on doit faire hériter une classe de type (ou d'une métaclasse déjà crée)
- On surdéfinira le constructeur __new__ de cette classe en faisant explicitement appel à type.__new__ via la fonction super()
- On associera la métaclasse dans la définition d'une classe par le mot clé metaclass

```
class Meta(type) :
    def __new__(cls, name, bases, attrs_dict) :
        super().__new__(name, bases, attrs_dict)
        special_attr = 100

class C(metaclass=Meta) :
    pass

C.special_attr # 100
c = C()
c.special_attr # 100 les attributs de classes sont des attributs d'instances dans les objets
```

Fonctionnalités Avancées

- Utilisation des métaclasses

- On utilise les métaclasses principalement pour la surdéfinition du constructeur
- Certaines libraires des framework python les utilisent (Django Forms et Django ORM par ex)
- On peut créer des design pattern avec (ex Singleton)
- Il est néanmoins conseillé de privilégier toute implémentation alternative
- Adage python :

  « Metaclasses are deeper magic than 99% of users should ever worry about. If you wonder whether you need them, you don’t - the people who actually need them know with certainty that they need them, and don’t need an explanation about why. »

DAWAN - Reproduction interdite

Gestion des packages tiers

- Rappels sur pip

```
$ pip install package # install globale de pakage

$ pip install --user package # install utilisateur de package

$ pip install ‘package==1.4’ # install version spécifique de package

$ pip install ‘package > 1.4, < 2’ # version de package acceptées

$ pip install ‘package ~= 1.4’ # version compatible avec 1.4 (1.* et ≥ 1.4)

$ pip install package –upgrade # upgrade package

$ pip uninstall package # désinstall

$ sudo pip install -U pip # upgrade pip
```


DAWAN - Reproduction interdite

Gestion des packages tiers

- Pipenv : installation, venv et dépendances

Le paquet pipenv associe l'installation de paquet (pip) en créant au besoin un environnement virtuel (venv) et un fichier au format toml pipfile des dépendances

```
$ pip install pipenv

$ pipenv install # création du fichier pipfile si absent ou installation des dépendances si présent + création venv si absent

$ pipenv shell # activation venv
...
$ pipenv install package
$ pipenv install requirements.txt # intégration à pipenv des paquets importés via pip
...
$ exit
```


DAWAN - Reproduction interdite

Gestion des packages tiers

- Pipenv : miscellanées

```
$ pipenv install package --dev # ce paquet ne sera pas installé par pipenv install
$ pipenv install --dev # installation des dépendances avec dev

$ pipenv lock # création de pipfile.lock de résolution des conflits de dépendances présents dans pipfile

$ pipenv install –ignore-pipfile # installe en se basant sur pipfile.lock optimisé

$ pipenv graph # représentation structurée en arbre des dépendances
```


DAWAN - Reproduction interdite

Gestion des packages tiers

- Recherche de packages avec pip

```
$ pip list # liste des paquets installés dans l'environnement courant

$ pip show package # documentation sur un paquet installé

$ pip search package# recherche 'package' dans les noms et
descriptions des paquets du dépôt distant

$ pip freeze > requirements.txt# imprime toutes les dépendances de
l'environnement courant dans le fichier requirements.txt

$ pip install -r requirements.txt # installation des dépendances dans
l'environnement courant
```


DAWAN - Reproduction interdite

Gestion des packages tiers

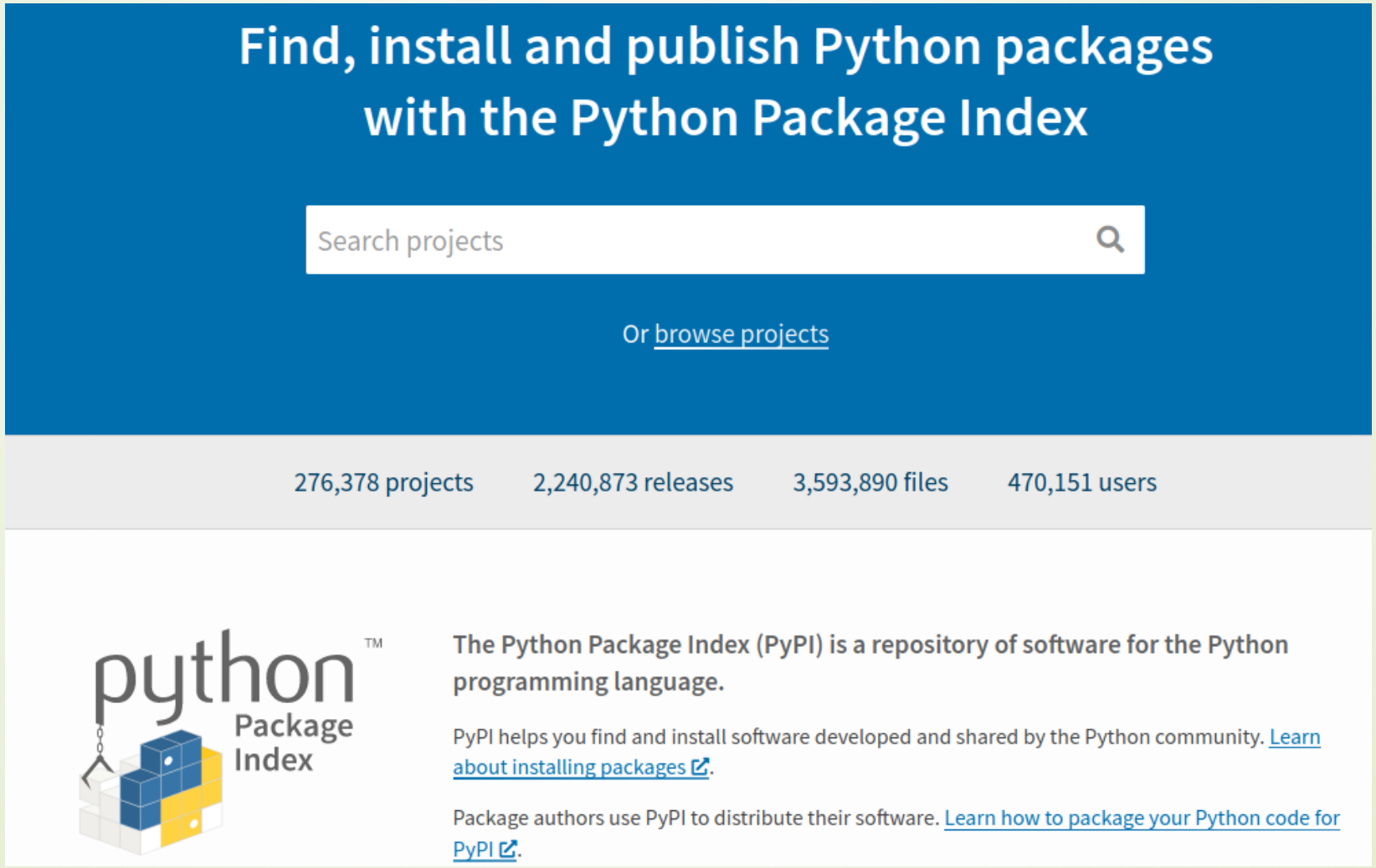
- Le dépôt Pypi

- Python Package Index : https://pypi.org
- c'est le dépôt officiel des packages tiers python sur lequel pointe pip par défaut
- https://pypi.org/simple est le point d'entrée par défaut de pypi pour pip
- Alternatives :
  - pip install --index-url http://local.url/path/to/my/repo/ package : dépôt privé
  - pip install -e install git+https://github.com/pathto/repo.git@branch : dépôt git
  - pip install -e path/to/package : dépôt local

DAWAN - Reproduction interdite

Gestion des packages tiers

- Le dépôt Pypi



Gestion des packages tiers

- Packager un projet

- Structure d'un package installable via pip

```
distribution_package
└── package
    └── __init__.py
```

- Fichiers et répertoires spécifiques à inclure
  - LICENSE : texte de la licence choisie (https://choosealicense.com/)
  - README.md : description au format MarkDown
  - setup.py : fichier de description du package en python exécuté par le module setuptools
  - tests : tests unitaires

```
distribution_package
├── LICENSE
├── package
│   └── __init__.py
├── README.md
├── setup.py
└── tests
```

- Génération d'une « distribution archive » gérable par pip

DAWAN - Reproduction interdite

Gestion des packages tiers

- Setup.py

```
import setuptools

with open("README.md", "r", encoding="utf-8") as fh:
    long_description = fh.read()

setuptools.setup(
    name="distribution_package", # nom de la distribution archive
    version="0.0.1",
    author="Example Author",
    author_email="author@example.com",
    description="A small example package",
    long_description=long_description,
    long_description_content_type="text/markdown",
    url="https://github.com/pypa/sampleproject",
    packages=setuptools.find_packages(),
    classifiers=[
        "Programming Language :: Python :: 3",
        "License :: OSI Approved :: MIT License",
    ],
    python_requires='>=3.6',
)
```

Gestion des packages tiers

- Distribution Archive

- Sur les versions récentes de pip, on préferera générer une « build distribution archive » au format wheel (.whl)
- Sinon on générera une « source archive » au format .tar.gz

```
pip install setuptools wheel

py setup.py sdist bdist_wheel

#résultat
dist/
  distribution_package-0.0.1-py3-none-any.whl # bdist_wheel : build distribution archive
  distribution_package.tar.gz # sdist : source archive
```


DAWAN - Reproduction interdite

Gestion des packages tiers

- Upload sur un dépôt pypi

- On va utiliser un dépôt de test : https://test.pypi.org/
- Création d'un compte : https://test.pypi.org/account/register/
- Création d'un Token d'API : https://test.pypi.org/manage/account/#api-tokens
- On utilise le paquet twine pour opérer l'upload

```
pip install twine

twine upload --repository testpypi dist/* # testpypi pointe sur https://test.pypi.org/
# un prompt demande le username et le Token API
...
# installation du package depuis le dépôt
pip install --index-url https://test.pypi.org/simple/ distribution_package
…
import package
```

- Si la procédure de test fonctionne, on la réédite avec https://pypi.org, sans l'option --repository

DAWAN - Reproduction interdite